AGROVISION

Intégration de Meta Omnilingual ASR

Documentation technique • MVP vocal multilingue • Préservation du dépôt cloné

OBJECTIF CENTRAL

Ajouter à AgroVision une description vocale des symptômes de la plante, transcrite par un moteur open source, sans réécrire ni dégrader les fonctionnalités existantes du dépôt Omnilingual ASR que vous avez cloné et adapté.

Décision	Périmètre retenu
Reconnaissance vocale	Meta Omnilingual ASR, auto-hébergé sur une machine distante adaptée.
Langues MVP	Français et anglais ; Ewondo à valider sur données réelles ; Cameroon Pidgin en mode expérimental.
Analyse agricole	Gemma reste séparé de l'ASR : image + texte transcrit et confirmé.
Principe de modification	Extension isolée, tests de non-régression, aucune suppression des fonctions upstream.

Document de conception et de sécurisation des modifications

Version 1.0 • 31 octobre 2026 • Basé sur la documentation publique du dépôt Meta et les ressources Common Voice.

IMPORTANT : le dépôt modifié n'a pas été fourni dans cette conversation. Ce document fixe le comportement attendu et la méthode de préservation ; il ne prétend pas décrire les fichiers réellement modifiés dans votre clone.

1. Résumé exécutif

AgroVision doit permettre à un agriculteur de joindre à une photo de plante une courte description vocale, dans une langue qu'il maîtrise. La voix est transcrite en texte ; l'agriculteur peut corriger la transcription ; le backend transmet ensuite la photo et le texte confirmé au module Gemma pour l'analyse agricole.

RÈGLE D'ARCHITECTURE

Omnilingual ASR ne diagnostique pas les maladies. Il transforme la parole en texte. Gemma analyse séparément la photo et la description confirmée. Le backend conserve la maîtrise des validations et du flux métier.

Décisions techniques

Élément	Décision / statut
Dépôt upstream	facebookresearch/omnilingual-asr ; conserver l'historique et la licence Apache 2.0.
Moteur ASR	Omnilingual ASR ; commencer par un benchmark des variantes CTC 300M v2 et LLM 300M v2.
Entrée audio	Enregistrement court, mono, normalisé selon le pipeline upstream ; limite produit recommandée : 30 s.
Limite upstream	Le README indique actuellement des fichiers de moins de 40 s pour les suites CTC et LLM standard.
Langue Ewondo	Le code ewo_Latn apparaît dans la liste des identifiants de langue ; cela ne garantit pas la qualité sur l'accent et les phrases agricoles.
Cameroon Pidgin	Aucun identifiant wes_Latn n'a été trouvé dans la liste vérifiée. À traiter comme langue expérimentale / adaptation à évaluer.
Hébergement	Service d'inférence séparé, sur serveur distant avec ressources adaptées ; pas d'inférence dans le navigateur.

Ce que le MVP doit prouver

- Un utilisateur peut enregistrer un audio court, recevoir une transcription et la corriger.
- La transcription corrigée est associée à la photo et transmise au flux Gemma déjà prévu.
- Si ASR échoue ou n'est pas disponible, la saisie manuelle et les fonctions antérieures restent opérationnelles.
- Les fonctions existantes du dépôt cloné passent toujours leurs tests après les changements.

2. Architecture cible AgroVision

L'intégration doit se faire par un adaptateur applicatif, sans coupler le code AgroVision à l'intérieur du moteur Meta. La séparation rend possible de désactiver la voix ou de remplacer le modèle plus tard sans toucher au diagnostic par image.

Étape	Responsabilité	Données / résultat
1. Interface	Capture photo et audio ; choix de langue ; durée limitée.	image, fichier audio, code langue choisi.
2. API AgroVision	Authentifie, contrôle taille/durée/type, conserve le brouillon.	requête protégée, ID de session.
3. Adaptateur ASR	Convertit le fichier au format attendu et appelle Omnilingual.	transcript, langue demandée, modèle.
4. Relecture	Présente la transcription à l'utilisateur.	texte corrigé ou rejeté.
5. Service Gemma	Reçoit photo + culture + texte confirmé.	analyse agricole structurée.
6. Backend	Enregistre les résultats, applique les règles métier.	dossier de diagnostic.

Flux de données



Contrat applicatif : le module vocal doit exposer une interface stable (par exemple transcribe_audio(file, language_code) -> TranscriptionResult) et ne pas exposer les classes internes Meta au frontend. Le résultat doit conserver le texte transcrit, le code demandé, le modèle et le statut ; ne pas inventer de score de confiance.

3. Langues et portée linguistique

La présence d'un identifiant de langue dans le dépôt signifie que le modèle connaît un code de conditionnement correspondant. Elle ne prouve pas que la transcription de messages spontanés, avec bruit de champ, accents locaux et vocabulaire agricole, est suffisamment précise.

Langue	Code vérifié	Décision MVP	Réserve
Français	fra_Latn	Cible prioritaire	Évaluer les accents et termes agricoles camerounais.
Anglais	eng_Latn	Cible prioritaire	Inclure les usages locaux et le bruit de terrain.
Ewondo	ewo_Latn	Pilote après benchmark	Code présent dans la liste ; qualité opérationnelle à mesurer.
Cameroon Pidgin	pas de wes_Latn trouvé	Mode expérimental	Nécessite tests spécifiques et potentiellement adaptation du modèle.

Ressources de données

Mozilla Common Voice, version 27.0 (publiée en septembre 2026), répertorie des jeux de données pour l'Ewondo et le Cameroon Pidgin. Ce sont des ressources utiles à l'évaluation et à la préparation d'une adaptation ; elles ne démontrent pas à elles seules une transcription fiable des descriptions spontanées.

Corpus	Données répertoriées	Licence / précaution
Ewondo (ewo)	7 991 clips ; 14,36 h enregistrées ; 13,61 h validées ; 31 locuteurs.	CC0-1.0 ; ne pas tenter d'identifier les locuteurs ; ne pas re-héberger ni redistribuer le corpus.
Cameroon Pidgin (wes)	9 417 clips ; 10,77 h enregistrées ; 10,31 h validées ; 18 locuteurs.	CC0-1.0 ; mêmes restrictions de non-identification et de non-redistribution.

POINT DE VIGILANCE

Les corpus Common Voice contiennent principalement de la lecture de phrases préparées. Pour AgroVision, créer aussi un petit jeu de voix spontanées : agriculteurs décrivant des feuilles, taches, jaunissement, flétrissement et évolution observée.

Positionnement dans le MVP

- Français et anglais : activer après tests de base et revue humaine.
- Ewondo : afficher comme pilote si les résultats locaux sont acceptables ; ne pas s'appuyer uniquement sur la liste de langues.
- Pidgin : afficher « expérimental » ; demander confirmation systématique et garder la saisie texte comme voie normale de secours.

4. Choix du modèle et contraintes d’exécution

Le dépôt publie plusieurs familles de modèles. Le choix final doit venir d’un benchmark sur les données AgroVision, pas uniquement de la taille ou des chiffres globaux de Meta.

Variante	Taille publiée	VRAM de référence	Utilité
omniASR_CTC_300M_v2	1,3 GiB	~2 GiB	Baseline légère et débit élevé ; pas de conditionnement explicite par code langue.
omniASR_LLM_300M_v2	6,1 GiB	~5 GiB	Accepte un code langue ; candidat intéressant pour Ewondo.
omniASR_LLM_7B_ZS	30 GiB	~20 GiB	Mode zero-shot avec exemples audio/texte ; très coûteux pour un MVP.

Les estimations de VRAM sont celles de la documentation du projet, mesurées en BF16, batch 1, audio de 30 secondes sur A100. Ce ne sont pas des exigences minimales universelles ni une garantie de performance sur un autre GPU.

Recommandation pragmatique

1. Benchmark initial : CTC 300M v2 pour disposer d’une baseline peu coûteuse.
2. Tester LLM 300M v2 pour les langues où le code de langue explicite améliore la transcription, notamment l’Ewondo.
3. Ne considérer LLM 7B ZS que pour une expérimentation Pidgin hors du chemin critique du MVP : il demande un serveur beaucoup plus puissant et exactement 10 emplacements de contexte audio/texte dans son mode d’inférence zero-shot.

Contraintes d’entrée

- Les modèles CTC et LLM standard acceptent actuellement des fichiers audio de moins de 40 secondes selon le README upstream. Limiter l’interface à 30 secondes pour garder une marge.
- Les modèles attendent une forme d’onde à 16 kHz ; utiliser le prétraitement fourni par le pipeline upstream plutôt que créer un second chemin de conversion non testé.
- Conserver l’audio original temporairement si nécessaire à la correction, avec durée de rétention définie. Ne pas le committer dans Git.
- Les modèles sont téléchargés au premier usage et mis en cache dans le répertoire fairseq2 ; prévoir un volume persistant côté serveur.

COÛT ET HÉBERGEMENT

Open source ne signifie pas calcul gratuit. Le code et les poids sont Apache 2.0, mais l’inférence distante nécessite une machine CPU/GPU, du stockage et de la bande passante. Ce document suppose un serveur séparé ; aucune exécution sur le PC personnel n’est requise.

5. Règles non négociables pour préserver le dépôt cloné

Le dépôt upstream a déjà été cloné et adapté. La priorité n'est donc pas de reconstruire le projet, mais de comprendre exactement les changements existants, de les préserver et d'ajouter AgroVision de façon isolée.

INTERDICTION PAR DÉFAUT

Ne pas supprimer, renommer, déplacer ou réécrire une fonction existante. Ne pas remplacer les fichiers upstream par une version simplifiée. Ne pas écraser les adaptations déjà réalisées. Toute modification d'un comportement existant exige une justification, un test avant/après et une validation explicite.

Procédure obligatoire avant toute édition

1. Créer une branche de travail dédiée, sans travailler directement sur main.
2. Capturer l'état actuel : git status, branche courante, commit HEAD, diff existant, fichiers non suivis.
3. Créer un commit ou un tag de sauvegarde du clone modifié avant de toucher au code.
4. Exécuter les tests existants et noter les résultats. Si certains échouent déjà, documenter ces échecs sans les attribuer aux nouveaux changements.
5. Cartographier les entrées publiques, exemples, scripts, CLI, configs, dépendances et tests du dépôt.
6. Écrire une note d'inventaire : fichiers upstream intacts, fichiers déjà adaptés, points d'extension possibles.

Stratégie d'intégration sûre

À privilégier	À éviter
Ajouter un adaptateur AgroVision isolé.	Modifier directement le cœur de chaque module upstream.
Ajouter un routeur/API séparé pour le service vocal.	Remplacer les fonctions publiques du pipeline.
Ajouter des tests et fixtures dans une zone dédiée.	Supprimer ou modifier les tests upstream pour faire passer la CI.
Épingler les versions de dépendances et documenter les conflits.	Mettre à jour toutes les dépendances sans analyse.
Ajouter un feature flag et un repli manuel.	Rendre la voix obligatoire pour utiliser le diagnostic.
Documenter les patchs et pouvoir les retirer.	Faire des modifications globales sans diff lisible.

Structure illustrative - à adapter après audit

```
repo/
  src/omnilingual_asr/     # upstream : préserver au maximum
  agrovision_asr/          # adaptateur / intégration spécifique (si compatible)
  tests/                   # tests upstream à conserver
  tests_agrovision/        # tests d'intégration dédiés
  docs/
    UPSTREAM_BASELINE.md
    AGROVISION_ADAPTATION.md
    REGRESSION_RESULTS.md
```

Cette arborescence est un exemple de séparation, pas une description de votre clone réel. Respecter le packaging et les conventions déjà présents.

6. Plan d’implémentation contrôlé

Phase	Travail	Critère de sortie
A. Audit	Examiner l’état modifié, le diff, les tests, dépendances, exemples et API.	Inventaire validé ; aucune modification de code.
B. Baseline	Exécuter tests upstream, quick-start, inference smoke test sur serveur.	Résultats enregistrés et reproductibles.
C. Adaptateur	Créer un wrapper isolé qui charge le modèle et normalise l’appel.	Un test unitaire vérifie l’interface sans charger le poids.
D. API vocale	Ajouter upload protégé, validation format/durée, réponse typée.	Audio test accepté ; erreurs invalides rejetées proprement.
E. Interface	Ajouter enregistreur, langue, transcription modifiable et saisie manuelle.	Le flux historique sans voix fonctionne toujours.
F. Gemma	Transmettre uniquement image + culture + texte confirmé au service Gemma existant.	Aucune dépendance de l’analyse image au service ASR.
G. Régression	Relancer tests upstream et tests AgroVision, comparer au baseline.	Pas de régression inexpliquée.

Exemple de test d’existence des langues

```
from omnilingual_asr.models.wav2vec2_llama.lang_ids import supported_langs

for code in ["fra_Latn", "eng_Latn", "ewo_Latn", "wes_Latn"]:
    print(code, code in supported_langs)

# Ne pas supposer que wes_Latn existe.
# Ce contrôle vérifie l'identifiant ; il ne mesure pas la qualité.
```

La documentation upstream montre une API ASRInferencePipeline et l’usage de codes du type eng_Latn pour le conditionnement des modèles LLM. Les modèles CTC n’acceptent pas le conditionnement de langue ni les exemples de contexte.

Exemple d’appel à encadrer

```
from omnilingual_asr.models.inference.pipeline import ASRInferencePipeline

pipeline = ASRInferencePipeline(model_card="omniASR_LLM_300M_v2")
texts = pipeline.transcribe([audio_path], lang=["ewo_Latn"])
transcript = texts[0]

# Encapsuler cet appel dans un adaptateur AgroVision.
# Le code de modèle reste configurable et testé.
```

Ne pas copier cet exemple sans vérifier que le modèle retenu, la version du package et les dépendances correspondent au commit du dépôt cloné. Le chargement du modèle doit être fait une fois par worker, pas à chaque requête.

Format de réponse stable côté AgroVision

```
{"transcript": "...", "requested_language": "Ewondo",
 "language_code": "ewo_Latn", "model_card": "...",
 "status": "success | needs_review | failed",
 "review_required": true}
```

7. Tests, qualité et non-régression

Tests obligatoires

Catégorie	Vérifications minimales
Upstream	Tous les tests existants ; exemples existants ; import des API publiques ; comportement du pipeline standard.
Adaptateur	Chargement du modèle ; modèle indisponible ; fichier absent ; langue inconnue ; sortie vide ; timeout.
Audio	Formats acceptés ; audio vide ; bruit ; clip > limite ; canal stéréo ; fréquence différente ; fichier corrompu.
Interface	Permission micro refusée ; arrêt/reprise ; correction ; annulation ; repli par texte.
Sécurité	Taille max ; extension/MIME vérifiés ; nom de fichier non fiable ; auth ; limites de requêtes ; stockage temporaire.
Intégration Gemma	Gemma reçoit uniquement la photo et le texte confirmé ; erreur ASR ne bloque pas le diagnostic manuel.

Protocole linguistique du MVP

- Constituer des enregistrements spontanés avec consentement, séparés par langue et par locuteur.
- Inclure des cas utiles : feuilles jaunes, taches brunes, trous, flétrissement, fruits abîmés, évolution depuis quelques jours.
- Tester plusieurs locuteurs, vitesses de parole, accents, bruits de vent, marché ou ferme, téléphones modestes.
- Faire transcrire la référence par un locuteur compétent ; comparer les prédictions à cette référence.
- Mesurer WER/CER, mais aussi la fidélité des mots agronomiques, noms de cultures et négations.
- Afficher une transcription à corriger si une erreur peut changer le sens du symptôme.

Critères d’acceptation proposés

Critère	Seuil de décision MVP
Régression upstream	0 nouveau test upstream cassé.
Parcours texte historique	100 % des parcours de référence restent utilisables.
Upload audio	Tous les fichiers hors limites sont rejetés sans crash.
Transcription	Évaluée séparément pour chaque langue ; aucun seuil global ne doit masquer un mauvais résultat local.
Pidgin / Ewondo	Pas de promesse de fiabilité avant revue par locuteurs et mesures.
Mode dégradé	Saisie manuelle utilisable à tout moment.

RÈGLE DE QUALITÉ

Une transcription plausible n’est pas nécessairement correcte. Si le texte contient un doute ou un terme agricole mal reconnu, l’utilisateur doit pouvoir le modifier avant qu’il soit transmis à Gemma.

8. Sécurité, confidentialité et licences

Données vocales

- Ne collecter l'audio qu'après une action explicite de l'utilisateur et l'affichage d'une courte information de confidentialité.
- N'enregistrer ni ne demander l'identité du locuteur pour la reconnaissance vocale.
- Limiter la durée de conservation de l'audio brut ; conserver la transcription nécessaire au diagnostic selon la politique du produit.
- Stocker les fichiers temporaires hors du dépôt Git et hors d'un répertoire statique public.
- Appliquer une authentification, une limite de taille, une limite de durée, un nettoyage automatique et une journalisation sans contenu vocal brut.

Licences et attribution

Élément	Licence / règle vérifiée
Code et modèles Meta Omnilingual ASR	Apache License 2.0 selon le dépôt officiel. Conserver la licence et les notices requises.
Common Voice Ewondo / Cameroon Pidgin	CC0-1.0. Le datasheet interdit d'essayer d'identifier les locuteurs et de re-héberger / redistribuer le corpus.
Poids téléchargés	Ne pas committer les poids dans Git ; conserver les identifiants de modèle, versions et provenance.
Modifications locales	Documenter chaque patch, son motif, son test et sa possibilité de retrait.

Exécution et secrets

- Le service ASR est derrière le backend ; aucun endpoint d'inférence non protégé ne doit être exposé directement au navigateur.
- Conserver les chemins de modèles/cache dans la configuration serveur ; ne pas supposer que le cache du conteneur est persistant.
- Ne pas envoyer les audios ni les transcriptions à une API propriétaire si l'exigence du challenge l'interdit.
- Séparer les logs techniques des données utilisateurs ; masquer les chemins privés et identifiants dans les logs.

9. Checklist de livraison et consignes au développeur

Checklist avant merge

☐ Le dépôt modifié a été sauvegardé avant toute intervention.
☐ Le commit/tag upstream ou baseline a été noté.
☐ Les modifications déjà présentes avant le travail sont identifiées et préservées.
☐ Les fichiers centraux upstream ne sont pas réécrits sans nécessité documentée.
☐ Le service ASR est isolé derrière un adaptateur.
☐ Le traitement audio est optionnel et dispose d'un repli texte.
☐ Les tests upstream passent comme avant (ou les échecs préexistants sont documentés).
☐ Les tests AgroVision et le smoke test réel passent.
☐ Les codes de langue sont vérifiés au runtime ; Pidgin reste expérimental.
☐ Aucun poids, audio privé, clé API ou dataset non redistribuable n'est ajouté à Git.
☐ Le diff final est relu : git diff --check, git diff --stat, git status.
☐ La documentation décrit les changements réellement réalisés, pas seulement les changements envisagés.

Instruction à donner à l'agent de développement

CONSIGNE COPIABLE

Tu travailles dans un dépôt Meta Omnilingual ASR déjà cloné et modifié pour AgroVision. Avant de changer quoi que ce soit, inspecte le dépôt réel, l'historique Git, les modifications locales, les tests et les fonctionnalités déjà opérationnelles. Ne suppose pas une arborescence. Ne remplace pas le code existant et ne supprime aucune fonction. Établis une baseline, puis ajoute l'intégration AgroVision dans une couche isolée et réversible. Toute modification d'un fichier upstream doit être minimale, justifiée et couverte par des tests avant/après. Garde la transcription vocale optionnelle, préserve le parcours texte existant, vérifie les identifiants de langue et traite le Cameroon Pidgin comme expérimental. À chaque étape, exécute les tests de non-régression et fournis le diff exact, la liste des fichiers touchés, les risques et les résultats. Si un changement risque de casser une fonctionnalité existante, arrête cette partie et propose un adaptateur au lieu de réécrire le code.

10. Références techniques et limites du document

Les éléments factuels suivants ont été vérifiés sur les ressources officielles et publiques consultées le 31 octobre 2026. Les performances réelles sur les voix du terrain AgroVision restent à mesurer.

Dépôt Meta Omnilingual ASR — https://github.com/facebookresearch/omnilingual-asr
README, installation, inference, architectures, langues supportées, limitations et liens vers les recettes.

Licence officielle — https://github.com/facebookresearch/omnilingual-asr/blob/main/LICENSE
Licence Apache 2.0 pour le code et les modèles du dépôt.

Liste des identifiants de langue — https://github.com/facebookresearch/omnilingual-asr/blob/main/src/omnilingual_asr/models/wav2vec2_llama/lang_ids.py
Contient ewo_Latn, fra_Latn et eng_Latn ; wes_Latn non trouvé lors de la vérification.

Documentation d'inférence — https://github.com/facebookresearch/omnilingual-asr/blob/main/src/omnilingual_asr/models/inference/README.md
Limite de durée, CTC, LLM conditionné par langue et mode zero-shot.

Common Voice 27.0 - Ewondo — https://mozilladatacollective.com/datasets/cmu6256q100mmo107kx3ckjvw
Métadonnées et licence CC0-1.0 du corpus Ewondo.

Common Voice 27.0 - Cameroon Pidgin — https://mozilladatacollective.com/datasets/cmu61lm5800kco107ohj0e507
Métadonnées et licence CC0-1.0 du corpus Cameroon Pidgin.

Annonce Meta — https://ai.meta.com/blog/omnilingual-asr-advancing-automatic-speech-recognition/
Présentation de l'approche multilingue et de l'extension aux langues sous-dotées.

Limite importante

Cette documentation ne confirme pas que votre clone actuel utilise une version précise, ni quels fichiers vous avez déjà modifiés. Elle ne doit donc pas être présentée comme un rapport d'audit du code ou comme une description « as built ». Pour produire une documentation fidèle aux adaptations déjà présentes, il faudra inspecter l'URL du fork ou une archive du dépôt modifié, puis remplacer les exemples génériques par le vrai inventaire des fichiers et des tests.

SYNTHÈSE

Préserver le dépôt d'abord. Ajouter l'ASR ensuite. Prouver la qualité linguistique sur de vraies voix camerounaises. Garder toujours le parcours de saisie manuelle fonctionnel.